

Experiment No. 7

Student Name: Swapnil Gaurkar

PRN: 25070521504

Experiment Title

Perform DOM traversal and updates; develop a to-do list app with add, edit, and delete features.

Software / Tools Required

- Visual Studio Code
- Google Chrome / Microsoft Edge
- HTML5
- CSS3
- JavaScript

Experiment Program Code

Files Used:

- to do.html
- to do.css
- to do.js

to do.html

```
<!DOCTYPE html>■
<html lang="en">■
<head>■
  <meta charset="UTF-8">■
  <meta name="viewport" content="width=device-width, initial-scale=1.0">■
  <title>My To-Do List</title>■
  <style>■
    * {■
      margin: 0;■
      padding: 0;■
      box-sizing: border-box;■
    }■
    body {■
      font-family: Arial, sans-serif;■
      min-height: 100vh;■
      display: flex;■
      justify-content: center;■
      align-items: center;■
      background: linear-gradient(135deg, #667eea, #764ba2);■
    }■
    .container {■
      width: 450px;■
      padding: 30px;■
      background: rgba(255, 255, 255, 0.15);■
      backdrop-filter: blur(15px);■
      border: 1px solid rgba(255, 255, 255, 0.3);■
      border-radius: 20px;■
      box-shadow: 0 20px 40px rgba(0, 0, 0, 0.25);■
    }■
    h1 {■
      text-align: center;■
      color: white;■
      margin-bottom: 8px;■
      font-size: 32px;■
    }■
    .subtitle {■
      text-align: center;■
      color: #eee;■
      margin-bottom: 25px;■
    }■
    .input-box {■
      display: flex;■
      gap: 10px;■
      margin-bottom: 20px;■
    }■
    #taskInput {■
      flex: 1;■
      padding: 14px;■
      border: none;■
      outline: none;■
      border-radius: 10px;■
```

```

    font-size: 15px;
  }

  #addBtn {
    padding: 12px 20px;
    border: none;
    border-radius: 10px;
    background: #22c55e;
    color: white;
    font-weight: bold;
    cursor: pointer;
  }

  #addBtn:hover {
    background: #16a34a;
  }

  .task-heading {
    display: flex;
    justify-content: space-between;
    color: white;
    margin-bottom: 12px;
  }

  #count {
    background: rgba(255, 255, 255, 0.2);
    padding: 5px 10px;
    border-radius: 20px;
    font-size: 13px;
  }

  ul {
    list-style: none;
  }

  li {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: white;
    padding: 12px;
    margin-bottom: 10px;
    border-radius: 10px;
  }

  .task-text {
    color: #333;
    max-width: 240px;
    word-wrap: break-word;
  }

  .buttons {
    display: flex;
    gap: 6px;
  }

  .edit,
  .delete {
    border: none;
    padding: 7px 10px;
    border-radius: 6px;
    color: white;
    cursor: pointer;
  }

  .edit {
    background: #f59e0b;
  }

  .edit:hover {
    background: #d97706;
  }

  .delete {
    background: #ef4444;
  }

  .delete:hover {
    background: #dc2626;
  }

  .empty {
    text-align: center;
    color: white;
    padding: 15px;
  }

  .footer {
    text-align: center;
    color: #ddd;
    margin-top: 20px;
    font-size: 13px;
  }
}
</style>
</head>
<body>
  <div class="container">
    <h1> My To-Do List</h1>

    <p class="subtitle">
      Organize your day, one task at a time.
    </p>

    <div class="input-box">
      <input

```

```

        type="text"
        id="taskInput"
        placeholder="What needs to be done?"
    >

</div>

<button id="addBtn" onclick="addTask()">
    + Add
</button>

</div>

<div class="task-heading">

    <h3>Your Tasks</h3>

    <span id="count">0 tasks</span>

</div>

<ul id="taskList">

    <li class="empty" id="emptyMessage">
        No tasks yet. Add one!
    </li>

</ul>

<div class="footer">
    Stay productive
</div>

</div>

<script>

let input = document.getElementById("taskInput");
let taskList = document.getElementById("taskList");
let count = document.getElementById("count");

function addTask() {

    let taskText = input.value.trim();

    if (taskText == "") {
        alert("Please enter a task!");
        return;
    }

    let emptyMessage = document.getElementById("emptyMessage");

    if (emptyMessage != null) {
        emptyMessage.remove();
    }

    let li = document.createElement("li");

    let span = document.createElement("span");
    span.textContent = taskText;
    span.className = "task-text";

    let buttons = document.createElement("div");
    buttons.className = "buttons";

    let editButton = document.createElement("button");
    editButton.textContent = "Edit";
    editButton.className = "edit";

    let deleteButton = document.createElement("button");
    deleteButton.textContent = "Delete";
    deleteButton.className = "delete";

    buttons.appendChild(editButton);
    buttons.appendChild(deleteButton);

    li.appendChild(span);
    li.appendChild(buttons);

    taskList.appendChild(li);

    input.value = "";

    updateCount();

    editButton.addEventListener("click", function() {

        let currentTask = this.parentElement.parentElement;

        let taskName = currentTask.children[0];

        let newTask = prompt(
            "Edit your task:",
            taskName.textContent
        );

        if (newTask != null && newTask.trim() != "") {
            taskName.textContent = newTask.trim();
        }

    });

    deleteButton.addEventListener("click", function() {

        let currentTask = this.parentElement.parentElement;

        currentTask.remove();

        updateCount();

        if (taskList.children.length == 0) {

```

```

    let message = document.createElement("li");
    message.textContent = "No tasks yet. Add one! ";
    message.className = "empty";
    message.id = "emptyMessage";
    taskList.appendChild(message);
  });
}

function updateCount() {
  let totalTasks = taskList.children.length;

  let emptyMessage = document.getElementById("emptyMessage");

  if (emptyMessage != null) {
    totalTasks = 0;
  }

  count.textContent =
    totalTasks +
    (totalTasks == 1 ? " task" : " tasks");
}

input.addEventListener("keypress", function(event) {
  if (event.key == "Enter") {
    addTask();
  }
});
</script>
</body>
</html>

```

to do.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
body {
  font-family: Arial, sans-serif;
  background: #f2f2f2;

  display: flex;
  justify-content: center;
  align-items: center;

  min-height: 100vh;
}
.container {
  width: 450px;
  background: white;

  padding: 30px;
  border-radius: 10px;

  box-shadow: 0 0 15px #ccc;
}
h1 {
  text-align: center;
  margin-bottom: 25px;
  color: #333;
}
.input-box {
  display: flex;
  gap: 10px;
}
#taskInput {
  flex: 1;

  padding: 12px;

  border: 1px solid #aaa;
  border-radius: 5px;

  font-size: 15px;
}
#addBtn {
  padding: 12px 20px;

  background: #2196f3;
  color: white;

  border: none;
  border-radius: 5px;

  cursor: pointer;
}
#addBtn:hover {
  background: #1976d2;
}
ul {
  list-style: none;
  margin-top: 20px;
}
li {
  display: flex;
  justify-content: space-between;
  align-items: center;

  padding: 12px;

  margin-bottom: 10px;

  background: #f5f5f5;
  border-radius: 5px;
}
li span {
  color: #333;
}
.buttons {
  display: flex;
  gap: 5px;
}
button {
  cursor: pointer;
  border: none;
  border-radius: 5px;
}
.edit {
  background: #ff9800;
  color: white;

  padding: 7px 12px;
}
.edit:hover {
```

```
    background: #f57c00;■  
}■  
■  
.delete {■  
  background: #f44336;■  
  color: white;■  
  ■  
  padding: 7px 12px;■  
}■  
■  
.delete:hover {■  
  background: #d32f2f;■  
}
```

to do.js

```
javascript
// Selecting DOM elements
const input = document.querySelector("#taskInput");
const addButton = document.querySelector("#addBtn");
const taskList = document.querySelector("#taskList");

// ADD TASK
addButton.addEventListener("click", function () {
  const taskText = input.value.trim();

  if (taskText === "") {
    alert("Please enter a task");
    return;
  }

  // Create new elements
  const li = document.createElement("li");

  const span = document.createElement("span");
  span.textContent = taskText;

  const buttonBox = document.createElement("div");
  buttonBox.className = "buttons";

  const editButton = document.createElement("button");
  editButton.textContent = "Edit";
  editButton.className = "edit";

  const deleteButton = document.createElement("button");
  deleteButton.textContent = "Delete";
  deleteButton.className = "delete";

  // Add buttons to button container
  buttonBox.appendChild(editButton);
  buttonBox.appendChild(deleteButton);

  // Add task and buttons to list item
  li.appendChild(span);
  li.appendChild(buttonBox);

  // Add list item to the webpage
  taskList.appendChild(li);

  // Clear input
  input.value = "";

  // EDIT TASK
  editButton.addEventListener("click", function () {
    // DOM Traversal
    const currentTask = this.parentElement.parentElement;

    // Access first child of list item
    const taskName = currentTask.children[0];

    const newTask = prompt(
      "Edit task:",
      taskName.textContent
    );

    if (newTask !== null && newTask.trim() !== "") {
      // Update DOM
      taskName.textContent = newTask.trim();
    }
  });

  // DELETE TASK
  deleteButton.addEventListener("click", function () {
    // DOM Traversal
    const currentTask = this.parentElement.parentElement;

    // Remove task from DOM
    currentTask.remove();
  });
});

// ADD TASK USING ENTER KEY
input.addEventListener("keypress", function (event) {
  if (event.key === "Enter") {
    addButton.click();
  }
});
```

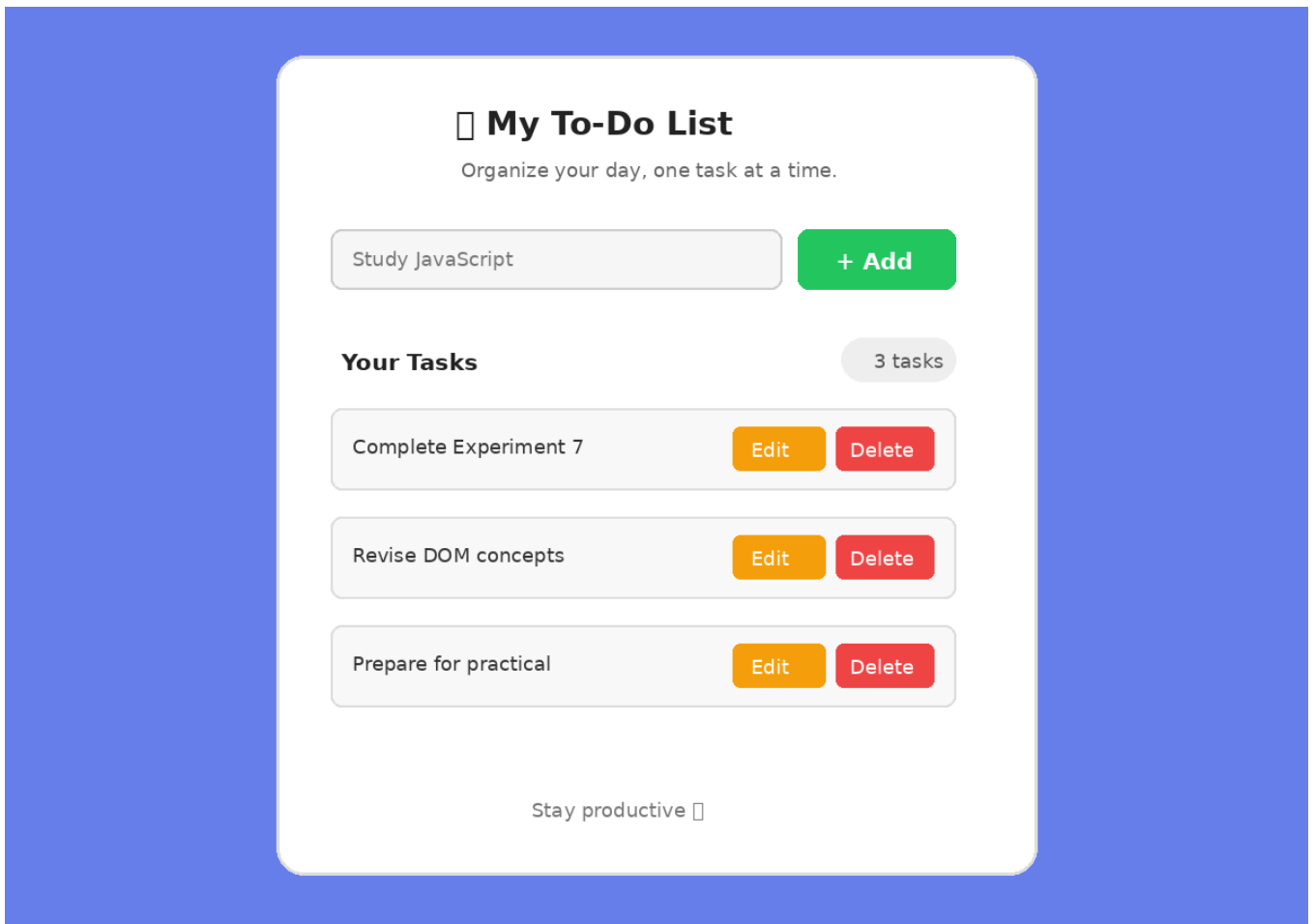
Application Description

The To-Do List application allows users to enter and manage daily tasks. JavaScript accesses DOM elements, creates new list items, and dynamically adds them to the webpage. Each task includes Edit and Delete buttons. The application also validates empty input and supports adding tasks using the Enter key.

Features Implemented

Feature	Description
DOM Selection	Selects input, button, and task list elements using DOM methods.
Add Task	Creates a new list item from user-entered text.
Edit Task	Uses DOM traversal and prompt() to update a task.
Delete Task	Removes the selected task from the DOM.
Input Validation	Prevents adding an empty task.
Keyboard Event	Allows adding a task by pressing the Enter key.
Dynamic Updates	Updates the task list without reloading the page.

Output



Output Summary

- Accepts a task through the input field.
- Adds tasks dynamically to the list.
- Allows tasks to be edited using the Edit button.
- Deletes tasks using the Delete button.
- Prevents empty tasks from being added.
- Supports the Enter key for quick task addition.

Result / Conclusion

The To-Do List application was implemented successfully using HTML, CSS, and JavaScript. The experiment demonstrated DOM traversal, DOM element creation, DOM updates, event handling, input validation, and dynamic task management.